

Characterization of Request and Token Energy Costs for LLM Inference Workloads on GPU Platforms

Prabhu Vellaisamy, Vanessa Lam, Shawn Blanton, and John Paul Shen
Electrical and Computer Engineering Department
Carnegie Mellon University
Pittsburgh, PA, USA

Abstract—Large language model (LLM) inference serving is priced by tokens, but GPU energy is consumed over inference windows. This accounting mismatch makes token-normalized metrics incomplete, since average output-token energy can decrease even when total request energy increases. We characterize this behavior with a decomposed energy model: a fixed one-time prefill with a fixed generation setup cost, while each output-token generation step adds marginal step energy. We evaluate this LLM inference energy model on NVIDIA H100 and H200 GPUs across dense and mixture-of-experts (MoE) models, reporting both request energy and token energy as functions of model type (M), phase (P), batch size (B), context length (C), and output length (N). For Llama-3.2-1B on H200 at batch-16 and context-4K, increasing output length from 10 to 512 tokens reduces token energy from 7.46 to 0.72 J/token while total batched inference-window energy increases from 1.19 to 5.93 kJ. Batching also reduces token energy, but the gain is context-bounded: at 10 output tokens, the batch-16 to batch-1 gain falls from $6.31\times$ at context-512 to $1.17\times$ at context-4K. MoE models amplify this effect: sparse routing and fragmented expert execution increase fixed energy at low concurrency, while batching spreads that energy across more generated tokens and substantially narrows the dense-vs.-MoE token-energy gap. These results show that energy-aware serving should jointly optimize both request energy and token energy, rather than only reducing per-token energy cost.

Index Terms—large language model inference, GPU energy efficiency, request energy, token energy, reasoning efficiency

I. INTRODUCTION

Large language model (LLM) inference is priced as a token-accounted service. Commercial APIs charge for input and output tokens [1], while serving systems report time-to-first-token (TTFT), time-per-output-token (TPOT), throughput, and “goodput” over token streams [2]–[5]. Production schedulers also make decisions based on token-visible quantities, such as batch size, prompt length, and generated output length. However, GPU energy is consumed during inference windows in which the GPU is allocated, kernels are launched, memory is accessed, synchronization occurs, and the CPU orchestrates the host/runtime to keep the GPU device actively utilized [6]–[8]. Therefore, there is an accounting mismatch: the service-visible unit is the token generated, but the hardware-visible unit for energy is the inference window. A scalar unit like Joules-per-token (J/token) is useful but paints an incomplete picture, since it does not reveal whether an efficiency gain comes from reducing total energy, or from spreading a fixed inference-window energy cost across more generated tokens.

For example, a longer output generation sequence can improve J/token while consuming more total Joules.

Recent industry-based economic model describes AI “tokenomics” through token utility, demand, supply, and monetization [9]. It identifies throughput per MW (TPS/MW) and cost per token as key supply-side metrics, which further motivates the understanding of what token-normalized metrics conceal. A token is an *economic accounting unit*, whereas GPU energy is incurred over a request execution window. Hence, an equal number of tokens need not impose the same physical costs, and a lower average cost per token need not indicate a lower total request cost. We ask: *When does a lower token-normalized energy cost represent a true reduction in GPU energy, and when does it reflect amortization of request-window energy?*

We describe each evaluated operating point using (M, P, B, C, N) , where M is the model and P denotes the inference phase: TTFT-oriented for the prefill-dominated window, or decode-oriented for a long-output window. Batch size (B) specifies how many requests share an inference window [3], context length (C) determines prompt-processing work and KV-cache state, and output length (N) determines how many autoregressive decode steps are executed. These parameters are already visible to serving systems, but a single token-normalized metric does not capture their energy impact.

Our energy model decomposes the total energy for each inference request into two components based on the two servicing phases: *prefill* phase processes the prompt, and *decode* phase generates output tokens [4], [5]. The central abstraction in this paper is a decomposed energy model for inference request energy (see Fig. 1 and Fig. 2):

$$E_{\text{request}(s)}(N) = E_{\text{fixed}} + N \times E_{\text{decode,step}} \quad (1)$$

where $E_{\text{request}(s)}(N)$ is the measured GPU energy of the complete inference window containing B requests, each generating N output tokens. E_{fixed} is the request-level energy independent of output length, and $E_{\text{decode,step}}$ is the marginal energy added per output token generation step for the complete batch. Conceptually,

$$E_{\text{fixed}} = E_{\text{prefill}} + E_{\text{decode,fixed}} \quad (2)$$

where E_{prefill} is the one-time energy used to process the prompt and construct the initial KV cache, and $E_{\text{decode,fixed}}$ represents generation-window setup and other costs that do not grow with output length.

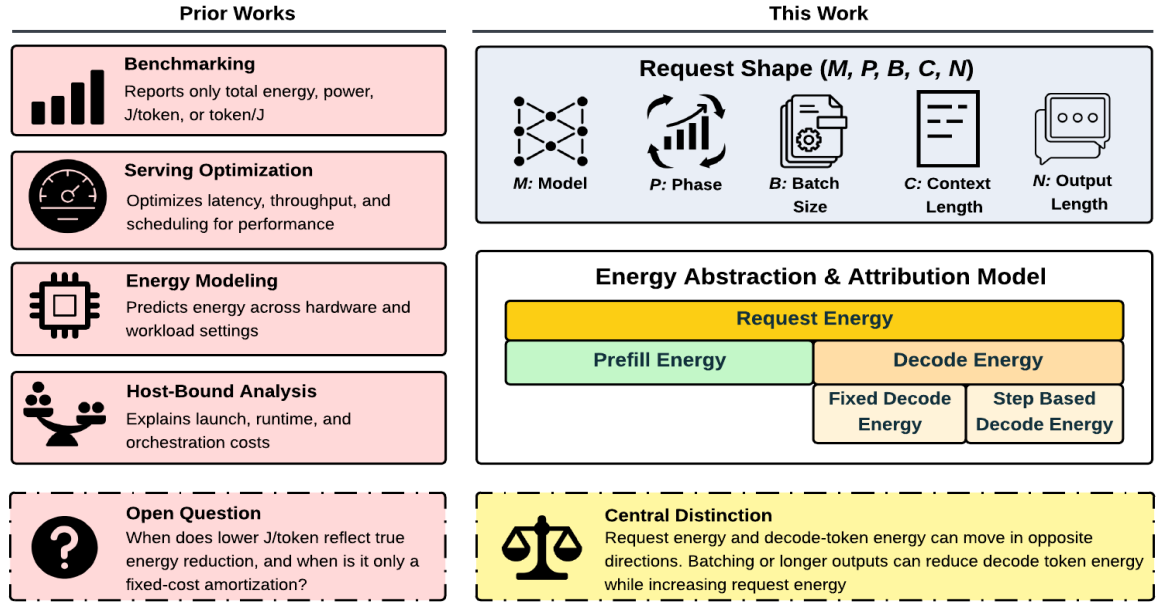


Fig. 1. Request-shape energy abstraction for LLM inference. Prior work reports aggregate energy, power, J/token, tokens/J, serving performance, energy models, or host-side overheads. This work focuses on when lower J/token reflects true energy reduction versus spreading fixed energy across more tokens. We characterize each request shape with five parameters, (M, P, B, C, N) , denoting model, phase, batch size, context length, and output length. Request energy decomposes into prefill and decode energy. Decode energy contains fixed energy paid once for the decode window and step energy added by each decode step. Our proposed energy abstraction explains why batching or longer outputs can reduce token energy while increasing total request energy. Both token energy and request energy must be considered in overall system optimizations.

Host/runtime orchestration, synchronization, model residency, and kernel-launch activity can contribute to both the fixed and step terms [7], [8]. One-time setup activity contributes to E_{fixed} , whereas residency energy and orchestration or launch activity that grow with inference-window duration are represented through $E_{\text{decode, step}}$. This decomposition holds model, batch size, initial context length, platform, and runtime fixed while output length varies. It is used as a local affine approximation over the measured output-length range, and for very long outputs, growing KV-cache traffic can cause marginal step energy to increase.

The corresponding average per-request energy is

$$E_{\text{req}} = \frac{E_{\text{request(s)}}(N)}{B} \quad (3)$$

This model explains why request energy and average token energy can move in opposite directions. We define average token energy as:

$$E_{\text{token}} = \frac{E_{\text{request(s)}}(N)}{B \times N} \quad (4)$$

This normalizes total batched request energy by $B \times N$ generated tokens and therefore includes fixed request-level energy amortized across those tokens. Increasing output length N can reduce energy per output token by amortizing fixed request costs, while still increasing total and per-request energy. Increasing batch size B can reduce token energy by sharing fixed costs across requests, although its effect on total and per-request energy depends on the request shape and platform. Energy-aware serving must therefore track both *request energy*

(E_{req}) and *token energy* (E_{token}). Modern serving systems already manipulate request shape for latency and throughput [3]–[5]. We study the same surface as an *energy-control* surface, asking when a shape reduces token energy by spreading fixed energy across more tokens and when this can lead to increasing the total request energy.

Prior LLM energy studies establish that inference energy depends on model, platform, runtime, and workload configuration, and commonly report average power, total energy, J/token, or tokens/J [10]–[16]. Energy modeling and serving-optimization studies further show that input length, output length, hardware choice, runtime configuration, and service level objective (SLO) constraints shift the energy operating point [17]–[22]. Host-side characterization studies show that framework dispatch, CUDA-library overhead, and kernel launch costs can dominate short decode regimes [7], [8], while execution-idle studies show that allocated GPUs can remain in elevated-power states even when visible activity is low [6]. These results motivate careful energy measurement, but do not separate request energy from token energy. We measure LLM inference energy using the GPU cumulative NVML energy counter [23], which gives total energy over the inference window. We evaluate dense and mixture-of-experts (MoE) models on NVIDIA H100 and H200 GPUs across eager and FlashAttention runtimes [24], TTFT-oriented and long-output request regimes, batch sizes B , context lengths C , and output lengths N up to 8K tokens. This paper makes the following contributions:

- **A fixed energy and step energy model.** We formulate measured request energy as fixed request-level energy plus

marginal step energy. The fixed term includes one-time prefill and fixed generation setup, while each output-token generation step contributes marginal energy. This model explains why token energy can decrease even when request energy increases (see Fig. 1 and Fig. 2).

- **A request-shape characterization of LLM inference energy.** We measure how analysis regime, batch size, context length, and output length affect request and token energy. Across platforms, H200 consistently lowers dense-model token energy, while MoE differences depend on model and request shape.
- **Evidence that output length and batching spread fixed energy.** We show that longer outputs and larger batches can reduce average token energy (E_{token}), but gains shrink with context length, and very long outputs can enter a regime where step energy dominates. We also show that MoE models are especially sensitive to batching, consistent with substantial routing, dispatch, and fragmented expert-execution overheads being amortized across request shape.
- **Attribution of floor- and kernel-active energy.** We combine NVML request-energy measurements with resident-floor and isolated-kernel replay measurements to explain why short, low-concurrency inference windows can be dominated by floor energy and orchestration rather than active kernel work.
- **Free batching for Perf/W.** We report prompt tokens/J to identify the free-batching regime, where additional batched prompt work fits into an already-open TTFT-oriented window and improves energy efficiency, and the point where larger contexts move the request into a compute-heavy regime. This exposes transitions that are not visible from $T_{N=1}$, the complete $N = 1$ window time, alone.
- **Utility-aware reasoning-budget characterization.** We evaluate Qwen3-8B, DeepSeek-R1-Distill-Qwen-7B, and DeepSeek-R1-Distill-Llama-8B on MATH-500 and ARC-Challenge under 64-, 256-, and 1024-token reasoning caps to show that utility/J-maximizing reasoning budget is model- and workload-dependent, and need not coincide with accuracy-maximizing budget.

II. BACKGROUND AND RELATED WORKS

A. LLM Inference Execution

LLM inference has two phases: *prefill* and *decode*. In prefill, the model processes the full input prompt in one forward pass and populates the key-value (KV) cache. As prompt tokens are processed together, prefill exposes substantial token-level parallelism and is typically dominated by large batched GEMMs [8]. In decode, the model generates one output token per forward pass. Decode is sequential across output tokens because each step depends on the KV state produced by previous steps. Its cost, therefore, grows with output length (N) and with the accumulated KV state, which scales with batch size (B) and context length (C).

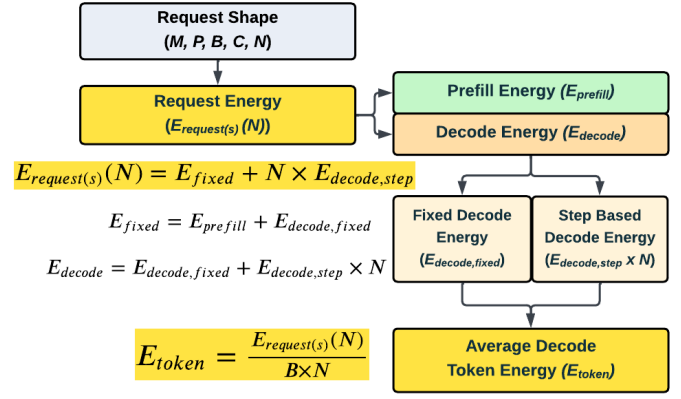


Fig. 2. Energy abstraction and attribution model: request shape-based energy decomposition by separating absolute request energy from token-normalized request energy.

Modern serving systems exploit this phase asymmetry through scheduling and memory management. PagedAttention virtualizes the KV cache into fixed-size blocks and enables continuous batching [3]. DistServe separates prefill and decode workers to reduce phase interference [4], while Sarathi-Serve chunks long prefills and schedules them alongside decode work [5]. These systems primarily optimize latency and throughput, but their packing decisions also change inference request energy.

B. Related Works

1) *LLM inference energy benchmarking*: Prior work has established GPU energy as a first-class metric for LLM inference. Words-to-Watts [13] measures inference energy across language tasks and serving stacks, while Chen et al. [10] report power and energy variation across GPU generations under matched workloads. TokenPowerBench [12] extends this direction with phase-aligned prefill/decode measurements across batch size, context length, inference engine, and parallelism. Wilhelm et al. [14] advocate energy-per-token as a first-class metric beyond accuracy, and Fernandez et al. [11] quantify how framework choice, batching, decoding strategy, quantization, hardware platform, and online/offline serving mode shift inference energy. These studies provide strong measurement foundations, but primarily report aggregate or token-normalized quantities such as average power, total joules, J/token, or tokens/J. Such metrics do not distinguish reductions in step energy from spreading fixed energy across more tokens.

2) *Token economics*: NVIDIA's tokenomics framework [9] links token utility and demand to infrastructure supply and monetization. It emphasizes model-system-software co-design and metrics such as cost per token and throughput per megawatt. This work is complementary: rather than modeling price or total cost of ownership, we isolate GPU energy and decompose the complete inference window into fixed and output-length-dependent components. This separation reveals that a favorable token-normalized metric is achieved by amortization rather than by a lower absolute energy.

TABLE I
COMPARISON OF THIS WORK AGAINST PRIOR LLM INFERENCE ENERGY STUDIES.

Work / Group	Focus	Phase	Batch	Ctx.	Out.	HW	MoE	Req./Token	Window
Energy benchmarks [10], [12], [13]	Power/energy measurement and token-normalized inference metrics.	✓	✓	✓	✓	✓	✓	×	×
Serving and optimization [11], [19]	Energy impact of frameworks, batching, decoding, hardware, and system control.	✓	✓	✓	✓	✓	✓	×	×
Input/output models [14], [17], [18]	Energy models and operating curves over input/output token counts.	✓	×	✓	✓	✓	✓	×	×
Host-side overhead [6]–[8]	Kernel-launch and dispatch decomposition, execution-idle high-power states.	✓	✓	✓	×	✓	×	×	✓
This Work	Abstraction of LLM inference energy costs by differentiating inference request energy and average token energy, and shows how schedulers should account for both for efficient inference service.	✓	✓	✓	✓	✓	✓	✓	✓

Legend: Phase = prefill/decode; Batch = batch size; Ctx. = context length; Out. = output length; HW = hardware/platform comparison; req./token = explicit separation of total request energy from token-normalized energy; Window = inference-window or decode-window amortization mechanism.

3) *Energy modeling and serving optimization:* A complementary line of work models LLM energy as a function of workload shape and serving configuration. Wilkins et al. [17] fit per-LLM energy and runtime models for offline energy-optimal scheduling on heterogeneous CPU-GPU systems. SweetSpot [18] derives an analytical efficiency model from Transformer compute and memory complexity and identifies non-linear input/output-length regimes. Cluster-scale work such as DynamoLLM [19] designs heterogeneous-instance and frequency-scaling policies under SLO constraints. These works show that energy efficiency depends strongly on workload shape, system configuration, and deployment policy, but they do not focus on cases where token-level efficiency improves while total request energy increases.

4) *Host overhead and execution-idle:* Prior CPU-GPU coupling work identifies host orchestration and kernel-launch overheads using kernel launch and queue time [7], while TaxBreak [8] decomposes inference overhead into Python translation, ATen dispatch, CUDA-library overhead, and kernel-launch components. Execution-idle studies further show that allocated GPUs can remain in elevated-power states even when visible kernel activity is low [6]. These results show that an inference window can carry fixed orchestration energy. However, they primarily expose host-bound behavior through latency and overhead decomposition, not through request-level energy or performance per watt.

C. This Work

This work makes the distinction between *request energy* and *token energy* the main object of characterization (Table I). We focus on request shapes where batching or longer outputs reduce token energy by spreading fixed energy across more tokens, but can still increase total request energy by extending the decode window. We capture this with a fixed energy and step energy model (Eq. 1) and use floor, kernel-active, and residual attribution to explain selected trends.

III. EXPERIMENTAL METHODOLOGY

Our methodology measures how request shape impacts GPU energy and token-normalized request efficiency. For each configuration, we vary the request-shape parameters and measure GPU energy over the corresponding inference window using the cumulative NVML energy counter.

A. Request Shape and Energy Metrics

We denote a measured configuration by (M, P, B, C, N) , with NVML energy measuring over a complete inference request. We define request energy as the total GPU energy consumed by each individual inference request:

$$E_{\text{req}} = \frac{E_{\text{request(s)}}(N)}{B} \quad (5)$$

$E_{\text{request(s)}}(N)$ is the measured energy of the complete batched request window. Its fixed term includes one-time prefill and fixed generation setup, while its output-length-dependent term captures marginal generation-step energy.

We focus on average token energy E_{token} as defined in Eq. 4, which normalizes total batched request energy by generated output tokens, $E_{\text{request(s)}}(N) = E_{\text{fixed}} + N \times E_{\text{decode,step}}$, which also includes prefill (Fig. 2).

B. NVML Energy Measurement

Dynamic power and energy telemetry on NVIDIA GPUs is exposed through the NVIDIA Management Library (NVML) [23]. We use two NVML interfaces. `nvmlDeviceGetTotalEnergyConsumption` reports cumulative energy consumption in mJ. Differences between synchronized readings at inference-window boundaries provide GPU energy and its associated circuitry during the measured window (E_{NVML}). `nvmlDeviceGetPowerUsage` reports power in mW averaged over a one-second interval. The cumulative energy-counter difference is used as the primary energy metric since the power samples averaging one-second

intervals cannot temporally resolve short decode windows, small batches, or transitions between GPU execution and host/runtime gaps [25], [26]. Hence, power readings are polled at a nominal 50 ms cadence as smoothed diagnostics through `pynvml` [27]. Calling NVML directly from the profiling process also avoids the startup and parsing overheads of repeated `nvidia-smi` queries.

C. Measurement Protocol

Each configuration performs warmup iterations followed by profiled iterations using `torch.profiler` [28], [29]. By default, we use $W = 10$ warmup runs and $R = 10$ profiled runs. Mean latency is reported over the profiled runs, and cumulative-counter energy is measured over the last five profiled runs and normalized per inference to maintain tractable measurement runtimes.

To ensure deterministic request shapes, prompts are synthesized as random token tensors drawn uniformly from each model’s vocabulary while excluding EOS IDs. The same input tensor is reused across warmup and measured iterations. Attention masks are all ones, so the requested context length C is the actual attended sequence length. Further, C is clamped to the model’s configured maximum, and EOS termination is disabled so that each request generates exactly N output tokens.

We use $N = 1$ as a TTFT-oriented, prefill-dominated configuration and $N > 1$ to measure complete fixed-length inference windows. The $N = 1$ measurement includes prompt processing, generation setup, and first-token production, while each $N > 1$ measurement includes the same one-time work followed by additional decode steps. Consequently, all request- and token-energy results are computed from the measured energy of the complete inference window. Floor and per-kernel replay measurements are used for diagnostic attribution and are described next.

D. Diagnostic Floor and Kernel Attribution

We use lightweight attribution to explain measured trends. First, we measure resident floor power from loading the model into GPU memory, initializing the runtime, synchronizing CUDA, and issuing no kernels. We sample GPU package power through `pynvml` at a nominal 50 ms cadence. When the cumulative NVML energy counter is available, we also take a longer idle measurement window counter difference as a cross-check and use the energy-counter-derived power as the primary resident-floor value. We use the measured resident floor energy as:

$$E_{\text{floor}} = P_{\text{floor}} \cdot T_{\text{window}} \quad (6)$$

This helps explain why short, low-concurrency decode windows can have high E_{token} . Second, we estimate kernel-active energy using isolated replay. Following the replay-based attribution strategy used in TaxBreak [8], we keep a database of unique kernels recorded in each model run along with their metadata (name, launch shape, input shape, and dtype). Each unique kernel signature is replayed in isolation in a child

process. The replay first runs a 500 ms warmup, then runs three 500 ms measurement windows whose boundaries are emitted by the child process and observed by the parent. The parent reads `nvmlDeviceGetTotalEnergyConsumption` at each boundary and also records `nvmlDeviceGetPowerUsage` as a smoothed diagnostic. Because Hopper power readings are averaged over one second, they are not used as an independent consensus channel for the 500 ms replay windows. Instead, replay reliability is evaluated from the cumulative energy-counter windows: at least two effective windows must remain, and the inter-window variation of the energy-derived net power must be at most 5%. For a kernel k , the replay-derived active energy contribution is:

$$E_{\text{active},k} = P_{\text{net},k} t_k \nu_k \quad (7)$$

where $P_{\text{net},k}$ is the energy-counter-derived active power above the local floor, t_k is the mean production duration, and ν_k is the production dispatch count. We define $E_{\text{active}} = \sum_k E_{\text{active},k}$. For these diagnostic cases, we report:

$$E_{\text{NVML}} = E_{\text{floor}} + E_{\text{active}} + E_{\text{residual}} \quad (8)$$

And hence, $E_{\text{residual}} = E_{\text{NVML}} - E_{\text{floor}} - E_{\text{active}}$. Because floor and isolated-replay estimates are not strictly additive under production overlap, the diagnostic residual may be signed. The residual is not redistributed to kernels. Because isolated replay cannot perfectly reproduce the production inference window, this attribution is used only to support interpretation of how fixed energy is spread across generated tokens.

IV. EXPERIMENTAL SETUP

A. Hardware Systems

We evaluate the workloads on two single-GPU NVIDIA Hopper-class platforms from a shared research cluster:

- **H100 platform:** Intel Xeon 8480C, 56 cores @ 2.0/3.8 GHz, PCIe Gen5, paired with an NVIDIA H100 SXM GPU with 80 GB HBM3 and 700 W nominal TDP.
- **H200 platform:** Intel Xeon Gold 6538Y+, 32 cores, paired with an NVIDIA H200 NVL GPU with 141 GB HBM3e and 600 W nominal TDP.

Table II summarizes the GPU-level differences relevant to LLM inference. H100 SXM and H200 NVL use the same Hopper generation, but they are not matched devices: H200 NVL has a larger and higher-bandwidth HBM3e subsystem, while the H100 SXM has a higher advertised peak Tensor-compute rate and a higher configurable TDP. All runs use one GPU and six CPU cores per GPU. Host-memory reservations vary from 32 to 192 GB to accommodate model- and trace-dependent profiler memory requirements.

We selected H100 and H200 as commercially available Hopper-generation platforms that are representative of current large-model deployments. The H200 provides greater memory capacity and bandwidth, features that affect decode energy by changing the cost of KV-cache traffic and long-context inference. This is a system-level comparison rather than an isolated memory experiment, since the two test platforms also

TABLE II

DIFFERENCES BETWEEN THE EVALUATED HOPPER GPU PLATFORMS. PEAK TENSOR THROUGHPUT ASSUMES SPARSITY. PRODUCT SPECIFICATIONS ARE FROM NVIDIA [30], [31]. PRIOR WORK SHOWS THAT ENERGY EFFICIENCY DEPENDS ON BOTH GPU ARCHITECTURE AND AI WORKLOAD UNDER POWER LIMITS [32], WHILE MEMORY BANDWIDTH AND DVFS CAN MATERIALLY AFFECT THAT RELATIONSHIP [33].

Feature	H100 SXM	H200 NVL
Compute generation	Hopper (GH100)	Hopper (GH100)
Peak BF16/FP16 Tensor	1,979 TFLOPS	1,671 TFLOPS
GPU memory	80 GB HBM3	141 GB HBM3e
Memory bandwidth	3.35 TB/s	4.8 TB/s
Maximum configurable TDP	700 W	600 W
Form factor	SXM	PCIe, dual-slot air-cooled

differ in host CPU, GPU form factor, and nominal TDP. We consequently attribute cross-platform energy differences to the complete platform configuration, and relate the observed trends to these features in the H100 vs. H200 results discussion.

B. System Software

We use a Python 3.13 environment with PyTorch 2.9.0+cu128, cuda12.6/toolkit module, and Transformers 4.57.3. Unless stated otherwise, runs use BFloat16 eager execution. FlashAttention-2 is enabled only for the corresponding sweep. Request energy is measured using NVML through the `pynvml` Python interface, while power telemetry is retained only as a diagnostic. Trace collection uses `torch.profiler`. Per kernel replay runs in a child process under `nsys profile -trace=cuda,nvtx` to validate CUDA dispatch and collect kernel activity.

C. LLM Workloads

We evaluate dense and mixture-of-experts LLMs in BFloat16. Dense LLMs include Llama-3.2-1B, Llama-3.2-3B, and Llama-3.1-8B [34]. MoE models include OLMoE-1B-7B and Qwen1.5-MoE-A2.7B, which differ in expert organization. OLMoE uses 64 routed experts per layer with top-8 routing and no shared experts, whereas Qwen1.5-MoE uses 64 experts per layer with 4 shared and 60 routed experts under top-4 routing. We use Hugging Face’s default per-expert independent matrix multiplications and disable token dropping. Prompts are synthesized to the target context length to eliminate tokenizer-induced length variation.

V. EXPERIMENTAL RESULTS AND OBSERVATIONS

First, we vary output length and batch size to illustrate how decode-window costs amortize and where that amortization ceases to hold. (Section V-A). We then compare H100 and H200 to separate platform scaling from request-shape structure (Section V-B), contrast prefill and decode as distinct operating regimes (Section V-C), and use per-kernel attribution to explain floor-dominated cases (Section V-D). We close by summarizing the scheduler-facing implications (Section V-E). Section VI then extends the analysis to utility-aware reasoning, examining how capped reasoning budgets trade task accuracy against

Request Energy Scaling on H200

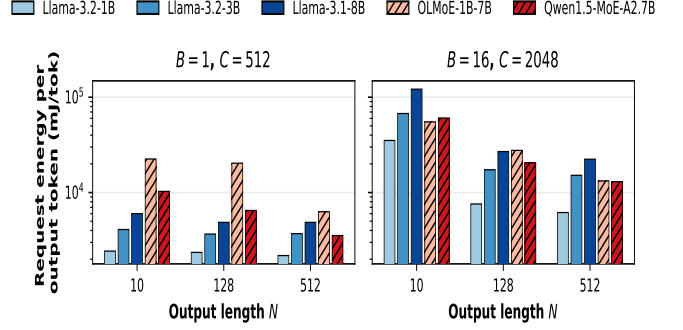


Fig. 3. Request energy scaling (mJ/token) across varying output generation lengths. The evaluation contrasts a low-load scenario (left) against a high-load scenario (right) on a single NVIDIA H200 GPU. The y-axis is log-scaled.

Request Latency Scaling on H200

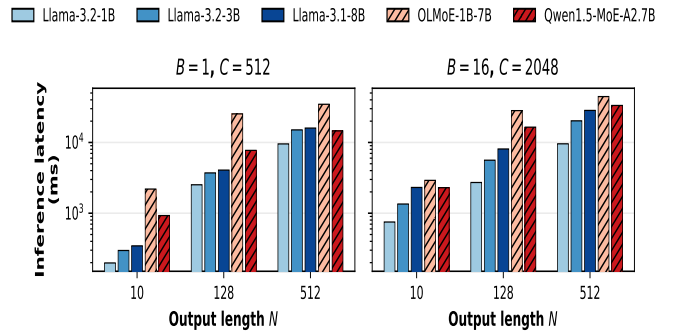


Fig. 4. Inference latency scaling (ms) across varying output generation lengths. The evaluation contrasts a low-load scenario (left) against a high-load scenario (right) on a single NVIDIA H200 GPU. The y-axis is log scaled.

request energy and utility per joule across models, benchmarks, and GPU platforms.

A. Output Length Is a First-Class Energy Axis

Fig. 3 plots token energy across output lengths $N \in \{10, 128, 512\}$ for two H200 operating points: low load ($B=1$, $C=512$) and high load ($B=16$, $C=2048$). Fig. 4 shows the corresponding latency results. Together, they show that output length plays a crucial role both as a performance axis and an energy-accounting axis.

1) Low-load amortization is modest for dense models:

At ($B=1$, $C=512$), increasing N from 10 to 512 tokens reduces token energy by only $1.1\times$ for Llama-3.2-1B ($2,431 \rightarrow 2,182$ mJ/token) and $1.2\times$ for Llama-3.1-8B ($6,011 \rightarrow 4,866$ mJ/token). This near-flat response means that extending the decode window does not substantially change the energy paid per generated token in this regime. The request remains low-concurrency and underutilized, so a large amount of fixed energy is spread across relatively little token work.

In contrast, the MoE models in Fig. 3 show a stronger reduction in token energy as N increases. This behavior is

TABLE III
REPRESENTATIVE H200 AFFINE FITS OVER $N \in \{10, 128, 512\}$.
RESIDUALS ARE $(E_{\text{fit}} - E_{\text{measured}})/E_{\text{measured}}$.

Model	B	C	E_{fixed} (J)	E_{step}/B (mJ/token)	Residual (%)		
					$N=10$	$N=128$	$N=512$
Llama-3.2-1B	1	512	12.11	2163.56	+38.79	-4.09	+0.26
Llama-3.2-1B	16	2048	276.78	351.06	-5.34	+2.53	-0.18
Llama-3.2-3B	16	512	30.63	437.17	-28.34	+5.95	-0.34
Llama-3.1-8B	16	512	83.73	604.84	-27.81	+7.38	-0.42

TABLE IV
BATCHING GAIN FOR COMPLETE H200 REQUESTS AT $N=10$, REPORTED AS
 $E_{\text{token}}(B=1)/E_{\text{token}}(B=16)$.

Model	$C=512$	$C=1024$	$C=2048$	$C=4096$
Llama-3.2-1B	$6.31\times$	$3.19\times$	$1.85\times$	$1.17\times$
Llama-3.2-3B	$4.67\times$	$2.66\times$	$1.70\times$	$1.22\times$
Llama-3.1-8B	$3.85\times$	$2.15\times$	$1.47\times$	OOM
OLMoE-1B-7B	$12.59\times$	$11.88\times$	$6.41\times$	$3.93\times$
Qwen1.5-MoE-A2.7B	$6.60\times$	$4.75\times$	$3.18\times$	$1.90\times$

consistent with a larger output-length-amortizable component arising from their larger total parameter footprint and the routing, dispatch, and fragmented expert-execution overheads identified by TaxBreak [8]. These mechanisms can contribute to both the request window and the per-decode-step costs. Therefore, the observed MoE scaling indicates greater amortization of low-concurrency overhead.

2) *High-load amortization is stronger*: At $(B=16, C=2048)$, the same output-length sweep yields a $5.7\times$ reduction for Llama-3.2-1B ($2,198 \rightarrow 386$ mJ/token) and a $5.4\times$ reduction for Llama-3.1-8B ($7,588 \rightarrow 1,398$ mJ/token). Here, fixed energy is spread across more generated tokens, so token energy improves even though the decode window consumes more total energy.

3) *A linear request-energy fit separates fixed energy and step energy*: For each fixed (M, B, C) configuration, we fit the measured complete inference-window energy at $N \in \{10, 128, 512\}$ to $E_{\text{request}}(N) = E_{\text{fixed}} + NE_{\text{step}}$. The intercept estimates energy independent of output length, while E_{step} is the incremental energy of one full-batch decode step. Table III reports E_{step}/B , the corresponding incremental energy per generated token.

The model captures the overall increase in request energy, but its relative error can be substantial at $N=10$, where the request is shortest. Table III further reports residuals at each measured length. We use the model only as a local decomposition over the measured range, not as evidence that marginal step energy is constant outside it.

4) *Batch gain is context-bounded*: Batch size (B) and output length (N) can both spread fixed energy across more generated tokens, but context length increases step energy. For Llama-3.2-1B at $N=512$, the $B=16$ token-energy advantage over $B=1$ is $11.2\times$ at $C=512$, but shrinks to $6.0\times$ at $C=2048$ and $3.7\times$ at $C=4096$. Table IV shows the same context-bounded pattern

TABLE V
LONG-OUTPUT REQUEST ENERGY PER OUTPUT TOKEN ON H200 AT $B=16$,
 $C=512$. VALUES ARE $E_{\text{request}(s)}(N)/(B \times N)$.

N	Llama-3.2-3B	Llama-3.1-8B
128	337 mJ/token	548 mJ/token
512	372 mJ/token	582 mJ/token
1024	455 mJ/token	691 mJ/token
4096	945 mJ/token	1,394 mJ/token
8192	1,600 mJ/token	2,358 mJ/token

for short-output decode at $N=10$, where the absolute gains are smaller because fewer output tokens are available to spread the fixed energy. As context grows, the KV state accessed during each decode step grows as well, moving the workload from a regime dominated by fixed energy toward one dominated by step energy. This trend is consistent with growing HBM demand: each decode step reads a larger KV cache as context length increases, so the memory subsystem becomes a larger component of step energy and leaves less fixed energy for batching to spread across tokens.

5) *Very long outputs reveal a second regime*: The linear fit is not intended as a global model for arbitrarily long generations. Extended request runs with $N \in \{1024, 4096, 8192\}$ show that token energy is not guaranteed to keep decreasing with output length (Table V). Llama-3.2-3B/H200 at $(B=16, C=512)$ reaches 337 mJ/token at $N=128$, but rises to 1,600 mJ/token at $N=8192$. Llama-3.1-8B/H200 at the same request shape rises from 548 mJ/token to 2,358 mJ/token. Output length therefore creates two regimes: one where more generated tokens reduce token energy by spreading fixed energy, and one where step energy dominates. The second regime is consistent with the growing cache and sustained active execution of long autoregressive generation, since each additional token both extends the decode window and increases the state accessed by subsequent steps. We therefore interpret the rise as step energy overtaking the fixed component.

6) *Batching amortizes MoE execution overheads*: At low load ($B=1, C=512, N=10$), OLMoE-1B-7B costs 22,476 mJ/token and Qwen1.5-MoE-A2.7B costs 10,277 mJ/token. These are $9.2\times$ and $2.5\times$ higher than their active-parameter-comparable dense counterparts, Llama-3.2-1B at 2,431 mJ/token and Llama-3.2-3B at 4,093 mJ/token. Prior TaxBreak analysis identifies routing, dispatch, and fragmented expert execution as important MoE overheads [8]. Our low-load energy gap is consistent with those mechanisms rather than active parameter count alone. The observed pattern is consistent with insufficient independent token work to fill the GPU effectively, leaving a relatively expensive under-filled execution window.

At high load ($B=16, C=2048, N=10$) the picture changes but does not collapse to a single ranking. OLMoE-1B-7B costs 3,440 mJ/token compared to Llama-3.2-1B at 2,198 mJ/token, leaving a $1.6\times$ active-parameter gap. Qwen1.5-MoE-A2.7B costs 3,781 mJ/token and becomes $1.11\times$ lower in token energy

than Llama-3.2-3B at 4,216 mJ/token. Relative to Llama-3.1-8B, a total-parameter-comparable dense reference at 7,588 mJ/token, OLMoE-1B-7B and Qwen1.5-MoE-A2.7B are $2.2\times$ and $2.0\times$ lower in token energy, respectively. The observed reduction is consistent with batching amortizing MoE routing and dispatch overheads, but the crossover depends on which dense baseline is used. Larger batches can provide more concurrent expert work and make fragmented expert kernels and dispatches less under-filled, allowing the GPU to remain active more consistently [8]. The model-specific crossover indicates that routing structure and execution regularity remain relevant alongside active-parameter count.

7) *Batching consistently improves token energy in the measured space:* Across measured (M, C, N) configurations on each GPU, increasing batch size from $B=1$ to $B=16$ reduces token energy. For OLMoE-1B-7B on H200 at $C=512$, $N=10$, token energy falls from 22,476 to 1,786 mJ/token ($12.6\times$). Llama-3.2-1B at the same shape falls from 2,431 to 386 mJ/token ($6.3\times$). Under-filled batches therefore leave substantial token-energy efficiency unrealized, although latency budget and queueing policy still determine whether batching is usable.

Key Insight. *Output length and batching reduce token energy only while fixed energy dominates. Context growth and very long outputs move the workload towards a regime where step energy dominates, so token energy and request energy must be considered together.*

B. Platform Energy Gains Depend on Model and Request Shape

Fig. 5 compares H100 and H200 at $N=10$ across evaluated (B, C) configurations. Short-output requests make fixed request-level energy more visible, allowing us to separate platform scaling from request-shape structure.

1) *H200 lowers token energy for dense models:* Across matched Llama-3.2-1B, Llama-3.2-3B, and Llama-3.1-8B configurations, H200 reduces token energy to $0.61\text{--}0.84\times$ of H100. For Llama-3.2-3B at $B=1$, $C=512$, token energy drops from 6,741 mJ/token on H100 to 4,093 mJ/token on H200, a $1.65\times$ reduction. The smallest dense-model gain is Llama-3.1-8B at $B=16$, $C=2048$, where the H200/H100 ratio is $0.84\times$. This is a higher-load regime on both platforms, the advantage from fixed energy contributes less to the total.

The shift in sampled diagnostic power is smaller than the energy shift. Across the plotted complete $N=10$ configurations, the median of the smoothed NVML power readings is 312.5 W on H100 versus 216.4 W on H200. Request shape still determines how long the GPU remains active, how many requests share the window, and how much KV state is accessed. The documented hardware differences make this a platform-level effect rather than a memory-only comparison. Both devices use the Hopper/GH100 generation, but H200 NVL provides 141 GB HBM3e and 4.8 TB/s bandwidth, compared with 80 GB HBM3 and 3.35 TB/s on H100 SXM. Conversely, H200 NVL has a lower advertised peak Tensor-compute rate and lower maximum configurable TDP [30], [31]. The H200 NVL’s lower

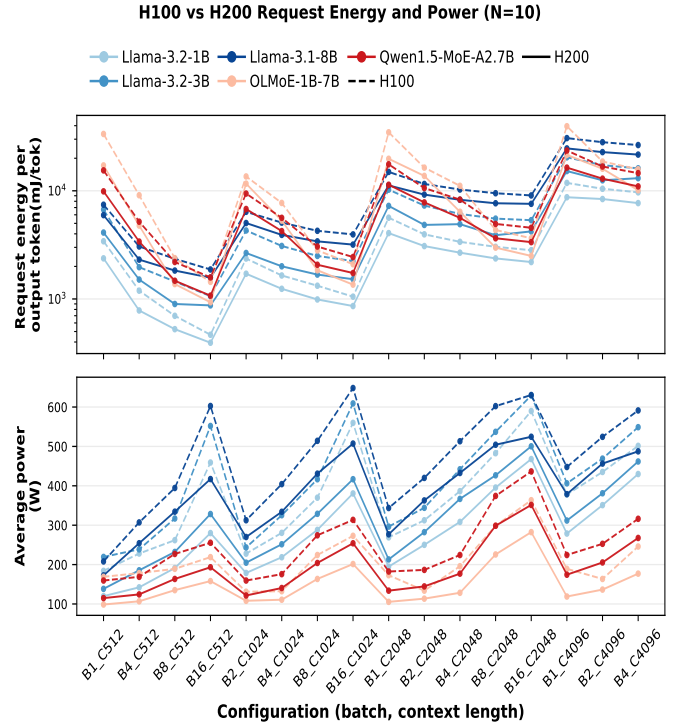


Fig. 5. Request energy per output token and power scaling on H100 (dashed) vs. H200 (solid) GPUs for output length $N = 10$. This figure illustrates the scaling behavior of dense and mixture-of-experts model architectures across varying batch sizes B and context lengths C .

configurable power envelope and different form factor are consistent with its lower sampled diagnostic power, while its higher HBM bandwidth may reduce execution time for memory-bound decode steps. Because the platforms also differ in host CPU and GPU form factor, the experiment cannot isolate the contribution of any individual feature. Prior H100/H200 studies likewise find that energy efficiency depends on workload, memory behavior, and power-management effects [32], [33].

2) *MoE platform gains are request-shape dependent:* At $(B=1, C=2048)$, OLMoE-1B-7B drops from 27,671 mJ/token on H100 to 22,035 mJ/token on H200, a $0.80\times$ ratio. Qwen1.5-MoE-A2.7B drops from 18,283 to 12,007 mJ/token, a $0.66\times$ ratio. The direction is not universal: at $(B=16, C=512)$, OLMoE-1B-7B is 1,786 mJ/token on H200 versus 1,462 mJ/token on H100, a $1.22\times$ ratio, despite lower sampled power on H200 (123 W versus 221 W). At $(B=16, C=2048)$, its token energy is nearly equal across platforms, 3,440 mJ/token on H200 vs. 3,521 mJ/token on H100.

3) *The request-shape trend is qualitatively preserved across the two measured platforms:* Although H200 generally shifts dense-model configurations downward in absolute energy, it does not alter the main structure of the energy surface. Low-batch and low-output decode remains expensive per token, batching still spreads fixed energy across more tokens, and context length still erodes the batch gain. Scheduling policies therefore need per-platform calibration for absolute energy, although we observe similar request-shape trends on the two

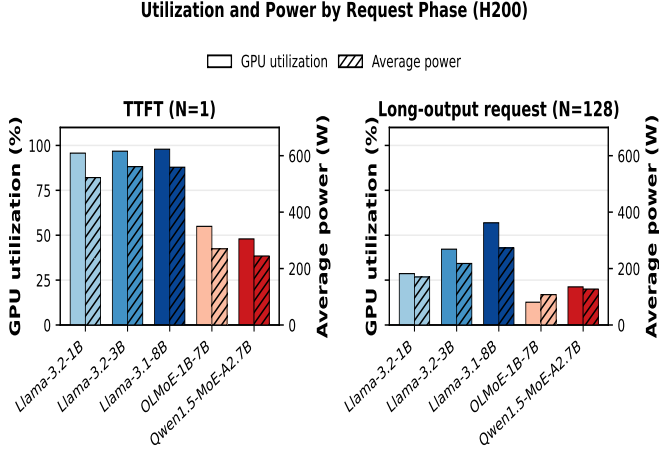


Fig. 6. TTFT-oriented ($N = 1$) and long-output ($N = 128$) inference operating points on H200 at $B = 4$, $C = 2048$.

evaluated platforms.

Key Insight. *H200 lowers token energy consistently for the measured dense models, while MoE gains depend on model and request shape. Platform calibration therefore remains necessary even when qualitative batching and context trends are preserved.*

C. TTFT and Long-Output Requests Occupy Distinct Operating Points

Fig. 6 compares TTFT-oriented ($N = 1$) and long-output ($N = 128$) inference windows at $B = 4$, $C = 2048$ on H200. The $N = 1$ window is prefill-dominated but also includes generation setup and first-token production, while the $N = 128$ window contains the same one-time work followed by additional decode steps. Therefore, this comparison characterizes two complete-request operating points.

1) *TTFT-oriented requests show high GPU utilization; long-output requests are lower-power but repeated:* At $B=4$, $C=2048$ on H200, the Llama-3.1-8B TTFT-oriented configuration ($N=1$) reaches 97.9% GPU active-time utilization, with sampled diagnostic power of 559.1 W, reflecting dense prompt processing over the 2,048-token context together with first-token production. The corresponding complete long-output request ($N=128$) has sampled diagnostic power of 273.0 W and 56.9% utilization, a 51.2% difference in the smoothed power readings. Llama-3.2-1B shows the same pattern: the $N=1$ configuration reaches 95.7% utilization with sampled diagnostic power of 522.4 W, while the $N=128$ request has sampled diagnostic power of 170.2 W and 28.6% utilization, a 67.4% difference. This behavior is consistent with prefill exposing substantial prompt-token parallelism, whereas the duration of a long-output request contains repeated autoregressive steps with lower GPU active-time utilization. Because NVML power telemetry on Hopper is averaged over one-second intervals, these power values characterize smoothed operating-point differences and are not used to compute request energy.

2) *Additional generation dominates long-output request energy:* At ($B=16$, $C=2048$) on H200, measured Llama-3.2-1B batched inference-window energy increases from 319 J at $N=1$ to 3,158 J at $N=512$. The additional 2,839 J relative to the TTFT-oriented configuration accounts for 89.9% of the complete $N=512$ batched inference-window energy. For Llama-3.1-8B, batched inference-window energy increases from 1,028 J to 11,455 J; the additional 10,427 J accounts for 91.0% of the $N=512$ total. Thus, lower sampled diagnostic power during long-output requests does not imply lower total energy: repeated autoregressive generation dominates their complete batched-window energy. These differences compare complete measured request windows and do not treat the $N=1$ measurement as isolated prefill energy.

3) *MoE TTFT-oriented windows show lower GPU utilization than dense-model windows:* OLMoE-1B-7B and Qwen1.5-MoE-A2.7B reach 54.9% and 47.9% GPU active-time utilization in the $N=1$ TTFT-oriented window, compared with 95.7-97.9% for the measured Llama-1B and Llama-8B configurations. Their sampled diagnostic power, 270 W for OLMoE-1B-7B and 244 W for Qwen1.5-MoE-A2.7B, is lower than the 559 W measured for Llama-3.1-8B. Lower power does not, by itself, imply better energy efficiency. The utilization results are consistent with lower, more irregular GPU activity due to expert routing and per-expert computation [8]. For mixed dense/MoE serving, this means TTFT-oriented power headroom and long-output energy efficiency can move differently across model families.

Key Insight. *TTFT-oriented and long-output request windows occupy different operating points. The former are prefill-dominated, while repeated generation with lower GPU active-time utilization determines much of the energy of long-output requests.*

D. The Energy Dynamics of Free Batching

Table VI shows the bounded free-batching region using complete $N = 1$ request windows. At $C=512$, increasing B from 1 to 4 nearly quadruples the prompt work while $T_{N=1}$ remains nearly constant at 25.18-28.73 ms. Prompt efficiency therefore rises from 85.49 to 220.32 prompt tokens/J.

The regime is bounded. At $B=8$, increasing C from 512 to 4096 raises $T_{N=1}$ from 45.27 to 1,175.80 ms and E_{NVML} from 33,905 to 686,698 mJ, while prompt efficiency falls from 120.81 to 47.70 prompt tokens/J. The sum of per-launch launch-to-start delays (TKLQT [7]) grows from 8,279 to 637,926 ms over the same span. Since individual queue intervals overlap, this sum is not elapsed overhead; its growth is consistent with a deeper device queue as execution becomes compute-heavy.

Table VII illustrates the endpoints using separate floor and replay-active shares: 61.1% and 13.6%, respectively, for the host-bound endpoint, versus 27.6% and 60.4% for the compute-heavy endpoint. The comparison shows a shift from an under-filled, floor-dominant window toward compute-heavy execution.

Key Insight. *Free batching is real but bounded.* $T_{N=1}$ shows when extra batch work is nearly latency-free, while prompt tokens/J shows whether it is energy-efficient. The replay

TABLE VI

LLAMA-3.2-1B/H100 TTFT-ORIENTED, PREFILL-DOMINATED TRANSITION FROM HOST-BOUND TO COMPUTE-HEAVY REQUEST SHAPES. $T_{N=1}$ IS THE COMPLETE $N = 1$ INFERENCE-WINDOW TIME. PROMPT TOKENS/J IS COMPUTED AS $(B \times C)/E_{\text{NVML}}$. THE TKLQT [7] SUM IS THE SUM OF PER-LAUNCH LAUNCH-TO-START DELAYS. SINCE QUEUE INTERVALS CAN OVERLAP, THE WALL-CLOCK TIME IS NOT ADDITIVE. BOLD TEXT MARKS CONFIGURATIONS WHERE $T_{N=1}$ REMAINS NEARLY CONSTANT AS BATCH SIZE INCREASES.

B	C	N	E_{NVML} (mJ)	TKLQT Avg.	TKLQT Sum	$T_{N=1}$ (ms)	Prompt tokens/J
1	512	1	5,988.9	8.05 μs	6.76 ms	25.18	85.49
2	512	1	6,810.4	24.38 μs	21.23 ms	25.41	150.36
4	512	1	9,295.6	668.23 μs	581.89 ms	28.73	220.32
8	512	1	33,905.2	9,507.70 μs	8,279.31 ms	45.27	120.81
8	1024	1	71,897.2	40,383.57 μs	35,166.01 ms	111.31	113.94
8	2048	1	191,210.2	145,693.76 μs	131,561.46 ms	325.74	85.69
8	4096	1	686,698.1	540,615.08 μs	637,925.79 ms	1,175.80	47.70

TABLE VII

LLAMA-3.2-1B/H100 DIAGNOSTIC FLOOR AND ISOLATED-REPLAY ESTIMATES FOR HOST-BOUND AND COMPUTE-HEAVY TTFT-ORIENTED, PREFILL-DOMINATED CONFIGURATIONS. FLOOR AND REPLAY-ACTIVE SHARES ARE NORMALIZED BY MEASURED NVML REQUEST ENERGY AND ARE REPORTED SEPARATELY BECAUSE THE DIAGNOSTIC ESTIMATES ARE NOT STRICTLY ADDITIVE.

Request Regime (Configuration)	TKLQT Avg (μs)	E_{NVML} (mJ)	Floor share	Replay-active share
Host-bound $B=1, C=512, N=1$	8.05	5,988.9	61.1%	13.6%
Compute-heavy $B=8, C=4096, N=1$	540,615.08	686,698.1	27.6%	60.4%

estimates show when the energy budget shifts from floor energy toward active kernel work.

E. Scheduler Implications and Energy-Aware Serving Design

The preceding results translate into three scheduler-facing rules: calibrate by model and runtime, choose batch size as a function of request shape, and track request-level cost rather than relying solely on the sampled power or average token-energy metric.

1) *Model and runtime effects require calibration:* Model size alone is not a reliable energy predictor. Across Llama/H200 at ($B=1, C=512, N=10$), Llama-3.2-3B has $3\times$ the parameters of Llama-3.2-1B but costs only $1.68\times$ more token energy (4,093 vs. 2,431 mJ/token), while Llama-3.1-8B has $8\times$ the parameters but costs only $2.47\times$ more. The same sub-linear trend remains at higher load: at $B=16, C=512, N=512$, the 8B/1B token-energy ratio is $3.18\times$. Runtime also shifts the energy surface. For Llama-3.2-3B on H100, FlashAttention-2 reduces token energy relative to eager by 13.4% at ($B=1, C=512$), 31.6% at ($B=1, C=2048$), and 51.9% at ($B=16, C=2048$). The increasing benefit at larger context and batch is consistent with fused attention reducing intermediate-memory traffic and improving reuse in the GPU memory hierarchy, rather than changing the model’s arithmetic work [24]. Sched-

ulers, therefore, need measured model/runtime profiles rather than parameter-count heuristics.

2) *Batch targets should be shape-aware:* Moderate batching often captures much of the available token-energy gain, but the knee is model- and context-dependent. For Llama-3.2-1B/H200 at $C=512, N=10$, moving from $B=1$ to $B=4$ captures 79% of the $B=16$ gain. For OLMoE-1B-7B at the same shape, $B=4$ captures 81%. The additional value of batching is consistent with routing and dispatch, leaving more fixed energy to spread across generated tokens. Architecture changes the crossover point as well: OLMoE-1B-7B becomes cheaper per output token than Llama-3.1-8B at ($B=4, C=2048, N=10$), with 7,298 vs. 8,235 mJ/token, and the gap widens at $B=16$.

Table IV summarizes the broader trend. Dense models lose most of their batching gain as C grows, while the larger gains for MoE models are consistent with expert-dispatch costs leaving more fixed energy to spread across generated tokens. Thus, the batch target should be tuned by model family, context length, output length, and latency SLO.

3) *Request-level metrics are necessary:* The fixed component is large enough that sampled power and token-normalized metrics can be misleading if reported alone. For rows with floor attribution, the median floor share is 51.6% of measured request energy; in low-load short-decode regimes ($B=1, C=512, N=10$), the median rises to 79.1%. At high-load long-output shapes such as ($B=16, C=2048, N=512$), it falls to 31.0%. Smoothed diagnostic power also compresses important differences: the most token-efficient measured decode setting, Llama-3.2-1B/H200 at ($B=16, C=512, N=128$), consumes 189.6 mJ/token and has sampled power of 181 W, while the least token-efficient setting, Llama-3.1-8B/H100 at ($B=1, C=8192, N=10$), consumes 84,784 mJ/token and has sampled power of 516 W. A $447\times$ token-energy spread appears as only a $2.8\times$ spread in the smoothed power readings.

Table VIII summarizes the resulting policy implications: batching is most valuable when fixed energy dominates, and less valuable when long-context KV-cache traffic or very long output generation makes step energy dominate.

Key Insight. *The same generated-token count can impose very different energy costs depending on request shape. Sched-*

TABLE VIII
REQUEST-SHAPE POLICY SUMMARY FOR ENERGY-AWARE SERVING.

Regime	Observed Energy Behavior	Strategy for Scheduler Actions
Short-context dense decode	Fixed energy dominates; batching strongly lowers token energy.	Use moderate batching when latency allows; tune the target by model scale, architecture, context length, and SLO rather than assuming a universal batch size.
Long-context dense decode	KV-cache traffic raises step energy; batch gain erodes with context length C .	Avoid waiting too long for large batches; optimize latency and request energy jointly.
MoE decode at low concurrency	Low-concurrency MoE behavior is consistent with routing and fragmented dispatch overheads.	Batch MoE requests more aggressively than dense requests at the same context length.
Long-output decode	Token energy first improves as fixed energy is spread across more tokens, then can rise when step energy dominates.	Track both request energy and token energy; do not optimize tokens/J alone.
TTFT-oriented vs. long-output requests	TTFT-oriented windows are prefill-dominated; repeated generation dominates long-output request energy.	Track both TTFT-oriented and long-output request energy within the fixed-plus-marginal model.

ulers should therefore expose request energy, token energy, and performance per watt together, and treat batch size as an energy-control knob whose benefit depends on context length, output length, and model family.

VI. UTILITY-AWARE REASONING EVALUATION

The tokenomics economic model [9] separates the utility delivered by generated tokens from the infrastructure cost of producing them, with the latter depending on request shape. This motivates a complementary question: *Do additional reasoning tokens deliver utility proportional to their energy cost?* Prior work couples task accuracy with inference energy through accuracy- or intelligence-per-joule metrics [35] and characterizes the energy cost of test-time reasoning [36], [37]. Building on this framing, we operationalize utility as final-answer correctness and use utility per joule to compare controlled reasoning-token budgets.

We evaluate Qwen3-8B, DeepSeek-R1-Distill-Qwen-7B, and DeepSeek-R1-Distill-Llama-8B on 500 MATH-500 questions and 300 ARC-Challenge questions using greedy decoding. All three models are evaluated on both H100 and H200 GPUs. The prompt and decoding configuration are fixed within each model, and only the maximum reasoning budget is varied across 64-, 256-, and 1024-token caps. The final-answer token allowance remains fixed. Because a model can terminate reasoning before reaching its cap, the measured energy reflects the actual generated sequence rather than the nominal cap alone. Each configuration uses $B=1$ and is averaged over three runs. For every response, we record final answer correctness, reasoning and answer tokens, truncation status, and request energy from the cumulative NVML energy counter.

For each recorded response i , let $U_i \in \{0, 1\}$ denote final-answer correctness and E_i its measured request energy. We

define utility per joule as

$$\eta_U = \frac{\sum_{i=1}^R U_i}{\sum_{i=1}^R E_i}, \quad (9)$$

where R is the number of recorded responses. Thus, η_U measures aggregate correct responses per joule. Accuracy is computed over distinct benchmark questions, while the energy reported in Table IX is mean energy per request. Unlike tokens/J, utility/J accounts for whether the generated output successfully completes the task. Hence, a system can generate tokens efficiently while producing little additional benchmark utility.

Table IX reveals distinct model-workload reasoning regimes. For Qwen3-8B on MATH-500, increasing the reasoning cap from 64 to 1024 raises accuracy from 26.0% to 68.8% on H100 and from 25.4% to 68.2% on H200. However, request energy increases by $7.42\times$ and $7.51\times$, respectively, reducing utility/J by approximately $2.8\times$ on both platforms. The diminishing return is more pronounced when short-budget accuracy is already high. For Qwen3-8B on ARC-Challenge, the same increase in cap improves accuracy by only 4.0 % points on H100 and 4.3 % points on H200, while request energy increases by approximately $4.3\times$. Consequently, utility/J decreases by approximately $4.1\times$ on both GPUs.

An intermediate reasoning budget is beneficial when its accuracy gain is sufficient to offset the additional energy. On MATH-500, DeepSeek-R1-Distill-Qwen-7B achieves its highest utility/J at the 256-token cap on both platforms. Similarly, DeepSeek-R1-Distill-Llama-8B achieves its highest ARC-Challenge utility/J at 256 tokens, where the additional reasoning substantially improves accuracy relative to the 64-token cap. In contrast, DeepSeek-R1-Distill-Llama-8B remains poorly matched to MATH-500: although its accuracy increases at 1024 tokens, the gain is insufficient to compensate for the

TABLE IX

UTILITY-AWARE CAPPED-REASONING RESULTS AT $B=1$. ENERGY IS MEAN PER REQUEST AND UTILITY PER JOULE IS AN AGGREGATE RATIO. DS DENOTES DEEPSEEK-R1-DISTILL.

Benchmark	Model	Cap	H100 Acc.	H100 Energy (J)	H100 Utility/J	H200 Acc.	H200 Energy (J)	H200 Utility/J
MATH-500	Qwen3-8B	64	26.0	316.8	0.000821	25.4	245.8	0.00103
		256	39.0	1,024.7	0.000381	39.0	796.3	0.000490
		1024	68.8	2,351.2	0.000293	68.2	1,845.8	0.000369
	DS-Qwen-7B	64	9.4	640.7	0.000147	9.2	515.2	0.000179
		256	23.2	1,211.4	0.000192	21.4	977.9	0.000219
		1024	38.0	3,148.8	0.000121	36.0	2,567.4	0.000140
	DS-Llama-8B	64	5.6	760.5	0.000074	5.6	551.4	0.000102
		256	6.2	1,365.0	0.000045	7.0	1,002.6	0.000070
		1024	13.4	2,805.1	0.000048	12.2	2,063.7	0.000059
ARC-Challenge	Qwen3-8B	64	90.0	272.6	0.003302	89.7	200.7	0.004467
		256	93.7	950.0	0.000986	93.3	696.1	0.001341
		1024	94.0	1,169.9	0.000804	94.0	856.3	0.001098
	DS-Qwen-7B	64	63.3	491.1	0.001290	63.3	394.0	0.001607
		256	73.0	1,023.0	0.000714	75.0	833.6	0.000900
		1024	78.7	2,035.9	0.000386	80.0	1,695.6	0.000472
	DS-Llama-8B	64	28.0	759.2	0.000369	29.7	546.3	0.000543
		256	57.3	1,355.7	0.000423	61.3	985.5	0.000622
		1024	78.3	2,276.8	0.000344	79.0	1,683.4	0.000469

additional request energy, and the 64-token cap remains utility-optimal. The utility-maximizing caps are therefore 64, 256, and 64 tokens for Qwen3-8B, DeepSeek-R1-Distill-Qwen-7B, and DeepSeek-R1-Distill-Llama-8B, respectively, on MATH-500, and 64, 64, and 256 tokens on ARC-Challenge. Notably, the 1024-token cap yields the highest accuracy for every model-benchmark pair, but does not maximize utility/J for any of them.

The platform comparison separates hardware efficiency from reasoning-budget efficiency. Across the evaluated configurations, H200 reduces mean request energy by 16.7-28.0 % relative to H100 and improves utility/J at every operating point, while the observed accuracy differs by at most 4.0 % points. Nevertheless, the utility-optimal cap is identical on H100 and H200 for every model-benchmark pair. Thus, greater hardware efficiency lowers the absolute energy cost of reasoning but does not eliminate the diminishing utility of additional tokens or change the preferred reasoning depth in these experiments.

Key Insight. *Reasoning budgets should be selected jointly by model, workload, and service objective rather than by accuracy or token count alone. Short caps are appropriate when accuracy has already saturated or when the model is poorly matched to the task, whereas moderate caps are justified when their marginal accuracy gain outweighs their additional request energy. Larger reasoning budgets should be reserved for settings in which accuracy requirements take priority over energy-normalized utility. Energy-aware serving systems should therefore expose accuracy, request energy, and utility/J together when selecting models and reasoning budgets.*

VII. SUMMARY AND CONCLUSION

This work presents an empirical characterization and energy model for GPU energy consumption during LLM inference. We evaluate this LLM inference energy model on NVIDIA H100 and H200 GPUs across dense and mixture-of-experts

(MoE) models, reporting both request energy and token energy as functions of request-shape parameters including: model type M , phase P , batch size B , context length C , and output length N . We characterize LLM inference energy as a request-shape problem rather than a static per-token cost. Using cumulative NVML energy measurements on H100 and H200 GPUs, we show that request energy and token energy can move in opposite directions: longer outputs and larger batches can reduce token energy by spreading fixed energy across more tokens, while increasing total request energy as the window extends. We also show that prefill has a bounded free-batching regime, where extra prompt work improves prompt tokens/J before larger contexts move execution into a compute-heavy regime. The scheduler-facing implication is that energy-aware serving should expose request energy, token energy, and performance per watt alongside latency, throughput, batch size, context length, and output length. Finally, we operationalize the *utility/J* metric. Reasoning tokens can lead to waste (saturating accuracy at certain token counts). Hence, models and the token counts can be configured for both high efficiency and low energy wastage.

REFERENCES

- [1] OpenAI, “API Pricing,” <https://openai.com/api/pricing/>, 2026, accessed: 2026-05-08.
- [2] V. J. Reddi *et al.*, “MLPerf inference benchmark,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 446–459.
- [3] W. Kwon *et al.*, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 611–626. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [4] Y. Zhong *et al.*, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 193–210. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>

- [5] A. Agrawal *et al.*, “Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 117–134. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/agrawal>
- [6] Y. Lei *et al.*, “The energy cost of execution-idle in gpu clusters,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.04745>
- [7] P. Vellaisamy, T. Labonte, S. Chakraborty, M. Turner, S. Sury, and J. P. Shen, “Characterizing and optimizing LLM inference workloads on cpu-gpu coupled architectures,” in *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2025, pp. 49–61.
- [8] P. Vellaisamy, S. Tripathi, V. Natarajan, S. S. Thenarasu, S. Blanton, and J. P. Shen, “Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.12465>
- [9] NVIDIA, “Tokenomics Guide: Case Studies and More,” Online documentation, 2026, accessed: 2026-08-25. [Online]. Available: <https://www.nvidia.com/en-us/solutions/ai/tokenomics-guide/>
- [10] Z. Chen *et al.*, “An empirical study on the power consumption of LLMs with different gpu platforms,” in *2024 IEEE International Conference on Big Data (BigData)*, 2024, pp. 8640–8642.
- [11] J. Fernandez, C. Na, V. Tiwari, Y. Bisk, S. Luccioni, and E. Strubell, “Energy considerations of large language model inference and efficiency optimizations,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 32 556–32 569. [Online]. Available: <https://aclanthology.org/2025.acl-long.1563/>
- [12] C. Niu *et al.*, “Tokenpowerbench: Benchmarking the power consumption of LLM inference,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 38, p. 32582–32590, Mar. 2026. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/40535>
- [13] S. Samsi *et al.*, “From words to watts: Benchmarking the energy costs of large language model inference,” in *2023 IEEE High Performance Extreme Computing Conference (HPEC)*, 2023, pp. 1–9.
- [14] P. Wilhelm, T. Wittkopp, and O. Kao, “Beyond Test-Time Compute Strategies: Advocating Energy-per-Token in LLM Inference,” in *Proceedings of the 5th Workshop on Machine Learning and Systems*, ser. EuroMLSys ’25. Association for Computing Machinery, 2025, pp. 208–215. [Online]. Available: <https://doi.org/10.1145/3721146.3721953>
- [15] S. Poddar, P. Koley, J. Misra, N. Ganguly, and S. Ghosh, “Towards sustainable NLP: Insights from benchmarking inference energy in large language models,” in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, L. Chiruzzo, A. Ritter, and L. Wang, Eds. Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 12 688–12 704. [Online]. Available: <https://aclanthology.org/2025.naacl-long.632/>
- [16] X. Liu *et al.*, “Position: LLM inference should be evaluated as energy-to-token production,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.11733>
- [17] G. Wilkins, S. Keshav, and R. Mortier, “Offline energy-optimal LLM serving: Workload-based energy models for LLM inference on heterogeneous systems,” in *Proceedings of the 3rd Workshop on Sustainable Computer Systems (HotCarbon)*, 2024.
- [18] H. Pizzini Cavagna *et al.*, “Sweetspot: An analytical model for predicting energy efficiency of LLM inference,” in *Proceedings of the 17th ACM/SPEC International Conference on Performance Engineering*. ACM, May 2026, p. 83–95. [Online]. Available: <http://dx.doi.org/10.1145/3777884.3797011>
- [19] J. Stojkovic, C. Zhang, I. Goiri, J. Torrellas, and E. Choukse, “Dynamollm: Designing LLM inference clusters for performance and energy efficiency,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1348–1362.
- [20] Q. Liu, D. Huang, M. Zapater, and D. Aienza, “Greenllm: Slo-aware dynamic frequency scaling for energy-efficient LLM serving,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.16449>
- [21] A. K. Kakolyris, D. Masouros, P. Vavaroutsos, S. Xydis, and D. Soudris, “Slo-aware gpu frequency scaling for energy efficient LLM inference serving,” 2025. [Online]. Available: <https://arxiv.org/abs/2408.05235>
- [22] M. M. A. Ifath and I. Haque, “Characterizing performance-energy trade-offs of large language models in multi-request workflows,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.09611>
- [23] NVIDIA Corporation, *NVML API Reference Guide*, NVIDIA Corporation, 2026, nVIDIA GPU Deployment and Management Documentation. [Online]. Available: <https://docs.nvidia.com/deploy/nvml-api/index.html>
- [24] T. Dao, “Flashattention-2: Faster attention with better parallelism and work partitioning,” in *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, Eds., vol. 2024, 2024, pp. 35 549–35 562. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/98ed250b203d1ac6b24bbcf263e3d4a7-Paper-Conference.pdf
- [25] A. McDaniel *et al.*, “Fine-grained power and energy attribution on amd gpu/apu-based exascale nodes,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.06056>
- [26] B. Tran, M. Maiterth, W. Shin, M. D. Sinclair, and S. Venkataraman, “Watchmen: Watching the watchers—high fidelity, flexible gpu energy modeling,” *arXiv preprint arXiv:2603.26435*, 2026.
- [27] nvidia-ml-py contributors, “nvidia-ml-py,” PyPI package. <https://pypi.org/project/nvidia-ml-py/>, 2026.
- [28] A. Paszke *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” *Advances in neural information processing systems*, vol. 32, 2019.
- [29] PyTorch Contributors, “PyTorch Profiler Tool,” Meta AI, Linux Foundation. https://pytorch.org/tutorials/recipes/recipes/profiler_recipe.html, 2026.
- [30] NVIDIA, “NVIDIA H100 Tensor Core GPU,” Product specifications, 2026, accessed July 2026. [Online]. Available: <https://www.nvidia.com/en-us/data-center/h100/>
- [31] NVIDIA, “NVIDIA H200 Tensor Core GPU,” Product specifications, 2026, accessed July 2026. [Online]. Available: <https://www.nvidia.com/en-us/data-center/h200/>
- [32] M. Mayr, S. Wind, L. Schröder, G. Hager, H. Köstler, and G. Wellein, “AI Application Benchmarking: Power-Aware Performance Analysis for Vision and Language Models,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.16164>
- [33] A. Ujeniya, J. Eitzinger, G. Hager, and G. Wellein, “Architectural Trade-offs in the Energy-Efficient Era: A Comparative Study of Power-Capping NVIDIA H100 and H200,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.11391>
- [34] A. Grattafiori *et al.*, “The llama 3 herd of models,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [35] J. Saad-Falcon *et al.*, “Intelligence per watt: Measuring intelligence efficiency of local ai,” *arXiv preprint arXiv:2511.07885*, 2025.
- [36] Y. Jin, G.-Y. Wei, and D. Brooks, “The energy cost of reasoning: Analyzing energy usage in LLMs with test-time compute,” *arXiv preprint arXiv:2505.14733*, 2025.
- [37] F. Oviedo *et al.*, “Energy use of ai inference, efficiency pathways, and test-time scaling,” *Joule*, p. 102430, Apr. 2026. [Online]. Available: <http://dx.doi.org/10.1016/j.joule.2026.102430>